

Chapter 2

Virtual Memory 와 Memory Management 심화

5 Lecture · 영상 86 분 · Ubuntu Server 26.04 + C
Observe → Understand → Implement → Analyze

L06 Process Virtual Address Space

L07 Virtual Address 와 Page Table

L08 TLB 와 Address Translation

L09 Page Fault 내부 동작

L10 mmap 과 Memory Mapped File

L06

Process Virtual Address Space

Chapter 2. Virtual Memory 와 Memory Management 심화
Process 가 보는 주소는 Physical Memory 의 주소가 아니다

영상 16 분

▶ 애니메이션 [C2-1 Process Address Space Explorer](#)

문제 상황

```
char *p = malloc(1024UL * 1024 * 1024);    /* 1 GB */
```

after malloc(1GB)	VmSize: 1051268 kB	VmRSS: 1548 kB
after touching 256MB	VmSize: 1051268 kB	VmRSS: 263692 kB

- 1GB 를 할당했는데 Memory 사용량은 그대로
- 건드린 만큼만 늘어난다
- " 할당 " 과 " 사용 " 은 다른 사건이다

복습 3 분 : Process Memory 구조

```

0x7fff...  [stack]           ↓
           [vvar] [vdso]
           mmap 영역       ↓   Shared Library, 큰 malloc
           [heap]         ↑   brk
           .data .bss
           .rodata
0x55...    .text
  
```

기존 과정 : Text / Data / BSS / Heap / Stack. Chapter 1: 이것은 ELF 의 LOAD 를 mmap 한 결과

→ Terminal 전환: `./addr_map`

maps 한 줄 = VMA 하나

```
5556711f9000-5556711fa000 r-xp 00001000 08:02 573648 /home/.../addr_map
└── 주소 범위 ────┘ 권한 offset 장치 inode 경로
```

```
task_struct → mm_struct → VMA { 범위, 권한, File, Offset } → VMA → VMA ...
```

mmap() 이 하는 일 = **VMA 하나를 만드는 것** . Physical Memory 는 아직 0 Byte

Heap 은 brk 로 , 큰 할당은 mmap 으로

```

== start                ([heap] 없음 )
== after malloc(1000)    55a6...5a000-55a6...7b000 rw-p [heap]      ← 132 KB
== after malloc(4MB)     [heap] 변화 없음 ,  big = 0x7f12535ff010
  
```

	작은 malloc	큰 malloc (≥128KB)
System Call	brk (가끔)	mmap (매번)
free 하면	glibc 내부로	즉시 munmap

→ Terminal 전환

Stack 은 저절로 커진다

start	[stack]	7fff880ad000-7fff880ce000	132 KB
depth 256	[stack]	7fff87cc6000-7fff880ce000	4128 KB

- 아래 방향으로 자란다 . System Call 을 부른 적이 없다
 - Kernel 이 Page Fault 처리 중에 VMA 를 확장 (L09)
- 한계 : ulimit -s (8MB). 넘으면 SIGSEGV

VSZ 와 RSS

	의미	malloc(1GB) 직후	256MB 건드린 뒤
VSZ (VmSize)	VMA 크기의 합 . 약속한 범위	+1GB	그대로
RSS (VmRSS)	지금 Physical Memory 에 있는 양	변화 없음	+256MB

Page 를 처음 건드리는 순간 4KB 씩 배정된다 = **Demand Paging**

"Memory 를 얼마나 쓰는가 " 는 RSS 로 본다

ASLR

```
$ ./addr_map | head -1 ; ./addr_map | head -1
text    main    0x5556711f9369
text    main    0x55d8c2a41369      ← 끝 3 자리만 같다 (Page 단위)
$ setarch -R ./addr_map | head -1
text    main    0x555555555369      ← 항상 같다 (gdb 안과 동일)
```

/proc/sys/kernel/randomize_va_space = 2

Process Address Space Explorer

화면 전환 → [animation.html](#)

시작 → malloc(1000) → malloc(4MB) → 깊은 재귀 → 1GB 할당 후 일부 사용

▶ 애니메이션 [C2-1 Process Address Space Explorer](#)

정리

주소는 Process 마다 따로 주어지는 **Virtual Address**

Kernel 은 주소 공간을 **VMA** 로 관리 . maps 한 줄 = VMA 하나

- brk / mmap / Stack 자동 확장

VSZ = 약속 , **RSS** = 실제 . Demand Paging

- ASLR 은 Page 단위

다음 : L07. CPU 는 이 가상의 번호를 어떻게 실제 위치로 바꾸는가

L07

Virtual Address 와 Page Table

CPU 는 Memory 에 접근할 때마다 주소를 변환한다

영상 18 분

▶ 애니메이션 [C2-2 Virtual Address Translator](#)

문제 상황

Process A 와 B 가 **같은 주소** 0x55555555369 를 쓴다 . 충돌하지 않는다

- 주소 공간은 128TB, 실제 Memory 는 4GB
- 변환표를 Byte 마다 만들면 표가 Memory 보다 커진다

변환은 모든 **Memory** 접근마다 일어나야 한다 → Software 로는 불가능

Page 단위로 변환한다

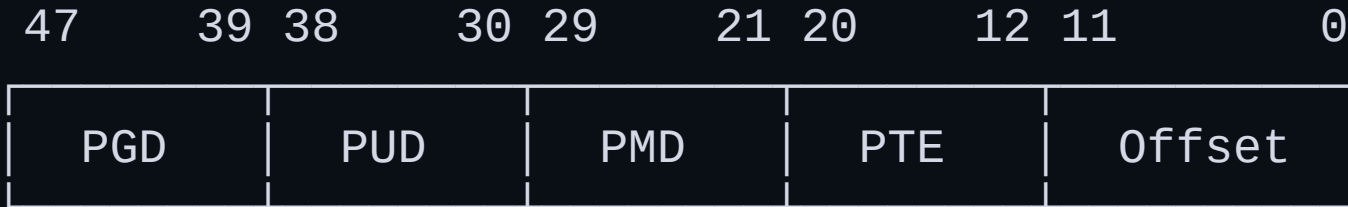
```

Virtual Address  = [      VPN      ][ Offset 12bit ]
                  | 변환              | 그대로
Physical Address = [      PFN      ][ Offset 12bit ]
  
```

Page	Virtual 주소 공간의 4KB 조각
Frame	Physical Memory 의 4KB 조각

maps 의 범위 , ASLR 의 끝 3 자리 , ELF 의 Align 0x1000, 권한의 단위 = 모두 4KB

48bit = 9 + 9 + 9 + 9 + 12



```
$ ./va_split 7ffd1234abcd
```

```
PGD 255    PUD 500    PMD 145    PTE 330    Offset 0xbcd
```

Table 하나 = Entry 512 개 × 8Byte = **4KB (Page 하나)**

→ Terminal 전환

왜 4-Level 인가

1-Level: $2^{36} \times 8\text{Byte} = \mathbf{512GB}$ (Process 마다)

```
CR3 → PGD — [170] → PUD → PMD → PTE Table → 실행 File, Heap
           — [171..253]   비어 있음 → 하위 Table 없음
           — [254] → ...                → libc, mmap
           — [255] → ...                → Stack
```

쓰지 않는 범위의 Table 은 만들지 않는다 . hello 의 Page Table 은 수십 KB

CR3 와 MMU

CR3: 현재 Process 의 PGD 가 있는 **Physical Address**

- Context Switch = CR3 교체 = 주소 공간 전체 교체

변환은 CPU 안의 **MMU Hardware** 가 수행

- Kernel 의 역할 : Table 의 내용을 채우고 CR3 를 설정하는 것까지

Virtual Address Translator

화면 전환 → [animation.html](#)

주소 입력 → 4 단계 Walk → Physical Address

▶ 애니메이션 [C2-2 Virtual Address Translator](#)

PTE: PFN + Flag

Flag	의미	만난 곳
P Present	Physical Memory 에 있다	Demand Paging
R/W	쓰기 허용	.rodata, RELRO, COW
U/S	Ring 3 접근 허용	Kernel 영역 보호
NX	실행 금지	W^X
A / D	접근됨 / 쓰여짐 (CPU 가 설정)	Reclaim, Writeback

VMA = 약속 (Kernel 이 읽음) / PTE = 현재 상태 (MMU 가 읽음)

확인 : pagemap, VmPTE, COW

```
page0 ( 건드리기 전 )    present=0
page0 ( 쓴 뒤 )          present=1  PFN 0x12e39d
512MB 건드린 뒤          VmPTE: 1072 kB          ← 131072 × 8 Byte
[child ] fork 직후       PFN 0x124dc9            ← Parent 와 같음
[child ] 쓰기 후         PFN 0x124dcc            ← 복사됨
```

fork = Page Table 만 복사 + R/W 를 0 으로 . 쓰는 순간 Page Fault 에서 복사

→ Terminal 전환

정리

- VPN → PFN, Offset 12bit 는 그대로
- $9 + 9 + 9 + 9 + 12$, 4-Level. 안 쓰는 범위의 Table 은 없다
- CR3 가 뿌리 , 변환은 MMU Hardware
- $PTE = PFN + \text{Flag}$. VMA 는 약속 , PTE 는 현재 상태
- fork 는 Page Table 만 복사한다

다음 : L08. Memory 접근마다 네 번을 더 읽으면 5 배 느려져야 한다 . 왜 그렇지 않은가

L08

TLB와 Address Translation

변환 결과를 기억해 두는 CPU 안의 작은 Cache

영상 16 분

▶ 애니메이션 [C2-3 TLB Hit / Miss Simulator](#)

문제 상황

```
mov (%rax), %ebx
```

① PGD ② PUD ③ PMD ④ PTE ⑤ Data ← Memory 접근 5 회

접근 횟수는 둘 다 2,097,152 회

dense (64 Pages) : 0.8 ms

sparse (262144 Pages) : 21.7 ms ← 28 배

같은 횟수의 Memory 접근인데 왜 수십 배 차이가 나는가

지역성과 TLB

- 프로그램은 방금 접근한 Page 를 곧 다시 접근한다 (Locality)
같은 Page 안의 4096 개 주소는 **같은 변환 결과**를 쓴다
- → VPN → PFN 결과를 기억해 두자



TLB 는 아주 작다

저장	VPN → PFN + 권한 Flag
크기	L1 수십 개 , L2 1,000~2,000 개
담당 범위	$64 \times 4\text{KB} = \mathbf{256\text{KB}}$, $2,048 \times 4\text{KB} = \mathbf{8\text{MB}}$
Miss 처리	x86-64 는 Hardware . Kernel 은 관여하지 않음
위치	Core 마다 따로

수 GB 를 쓰는 프로그램에 비해 TLB 가 기억하는 범위는 매우 작다 → **접근 Pattern** 이 중요

TLB Hit / Miss Simulator

화면 전환 → [animation.html](#)

Entry 4 개 , LRU. Step 하나 = Memory 접근 한 번

▶ 애니메이션 [C2-3 TLB Hit / Miss Simulator](#)

측정 : dense vs sparse

```
sum += p[i];           /* dense : 64 Pages 안에서만 */  
sum += p[i * PAGE];    /* sparse: 접근마다 다른 Page */
```

주의 : 이 차이에는 **CPU Data Cache Miss** 도 섞여 있다 . TLB 만의 효과는 Slide 9 에서 분리

Native Linux: `perf stat -e dTLB-load-misses` (VirtualBox 에서는 not supported)

→ *Terminal 전환*

Context Switch 와 TLB

TLB 의 내용은 특정 **Process** 의 변환이다 → Process 가 바뀌면 무효
Flush(전부 비움) 또는 **PCID**(Entry 에 주소 공간 Tag)

- Thread 간 전환 : 주소 공간이 같다 → TLB 유지
- Context Switch 의 진짜 비용 = 이후의 Cache / TLB Miss

munmap, mprotect → 다른 Core 의 TLB 도 지워야 한다 (TLB Shootdown)

Huge Page

	4KB	2MB
512MB 에 필요한 TLB Entry	131,072	256
Page Walk	4 단계	3 단계

```

4KB Page   : 30.1 ms      AnonHugePages:      0 kB
2MB (THP)  : 19.3 ms      AnonHugePages: 524288 kB
  
```

`madvise(p, len, MADV_HUGEPAGE)` · 접근 주소와 순서가 같으므로 차이는 변환 비용

→ Terminal 전환

정리

- TLB = VPN → PFN 결과의 Cache. Hit 면 Memory 접근 0 회
- 담당 범위가 작다 → 접근 Pattern 이 성능을 좌우
- TLB Miss 는 Hardware, Page Fault 는 Kernel
- Process 전환은 TLB 를 무효화 , Thread 전환은 유지
- Huge Page = Entry 하나의 범위를 512 배로

다음 : L09. "PTE 없음 → Page Fault → Kernel" 경로

L09

Page Fault 내부 동작

Page Fault 는 오류가 아니라 Linux 가 일부러 사용하는 정상 동작이다

영상 18 분

▶ **애니메이션** [C2-4 Page Fault Walkthrough](#)

문제 상황

```
첫 번째 쓰기 (10000 Pages) : minor +10000  
두 번째 쓰기 (10000 Pages) : minor +0
```

```
$ ./major_fault      (Cache 비운 직후)    312.6 ms  
$ ./major_fault      (다시 실행)          1.0 ms
```

- 처음 건드릴 때마다 Kernel 에 들어간다
- 같은 프로그램인데 첫 실행만 300 배 느리다

언제 발생하는가

MMU 가 본 것	예
Present = 0	malloc 후 첫 접근 , 아직 안 읽은 File Page
쓰기인데 R/W = 0	COW, .rodata 에 쓰기
Ring 3 인데 U/S = 0	Kernel 주소 접근
실행인데 NX = 1	Data 를 실행

CPU 가 남기는 것 : **CR2** = 주소 , Error Code = 읽기 / 쓰기 , User/Kernel

처리 후 같은 명령을 다시 실행한다

Handler 의 질문 : "VMA 는 뭐라고 하는가 "

```
#PF (CR2)
├ VMA 없음 - Stack 바로 아래 → VMA 확장
│   └ 그 외 → SIGSEGV (SEGV_MAPERR)
├ VMA 권한과 불일치 → SIGSEGV (SEGV_ACCERR)
└ 적당한 접근 → handle_mm_fault()
    Anonymous 첫 읽기 → Zero Page          Minor
    Anonymous 첫 쓰기 → 새 Frame           Minor
    File → Page Cache 에 있음              Minor
           없음 → Disk 에서 읽음           Major
    쓰기 + R/W=0 + VMA rw- → COW          Minor
→ PTE 갱신 → 같은 명령 재실행
```

Page Fault Handler = **현재 상태 (PTE) 를 약속 (VMA) 에 맞게 고치는 곳**

Page Fault Walkthrough

화면 전환 → [animation.html](#)

Minor → Major → COW → Invalid

▶ 애니메이션 [C2-4 Page Fault Walkthrough](#)

첫 접근과 Zero Page

mmap(256MB) 직후	RSS	1420 kB	
모든 Page 읽기	RSS	1744 kB	minor +65548
모든 Page 쓰기	RSS	263888 kB	minor +65536

읽기만 하면 Kernel 은 Frame 을 주지 않는다 . 하나뿐인 **Zero Page** 를 읽기 전용으로 공유

- 쓰는 순간 다시 Fault → 그제서야 진짜 Frame

→ Terminal 전환: `./fault_count`, `./zero_page`

Minor vs Major

	Minor	Major
조건	내용이 이미 Memory 에 있거나 새로 만들면 됨	Disk I/O 필요
비용	수 μ s	수백 μ s ~ 수십 ms
Process	계속 실행	I/O 완료까지 Sleep

```
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
```

File 은 Home Directory 에 . 26.04 의 /tmp 는 tmpfs

→ *Terminal 전환*

COW Fault

```
[child] 10000 Pages 읽기 : minor +0  
[child] 1000 Pages 쓰기 : minor +1000 ← 쓴 Page 만 복사
```

Redis 의 fork Snapshot: Child 는 fork 시점의 Memory 를 보며 저장 , Parent 는 계속 처리

SIGSEGV 의 두 종류

```
$ ./segv_info null
SIGSEGV  si_addr=0x10          SEGV_MAPERR  VMA 가 없음
$ ./segv_info ro
SIGSEGV  si_addr=0x559cb498d130 SEGV_ACCERR  VMA 는 있지만 권한이 없음
```

si_addr = CR2. 작은 값 (0x10, 0x18) 이면 NULL 구조체 포인터의 멤버 접근을 의심

Handler 에서 return 하면 같은 명령이 재실행되어 무한 반복

측정 도구

```
/usr/bin/time -v ./app          # Major / Minor page faults
perf stat -e page-faults,minor-faults,major-faults ./app      # Software Event: VirtualBox 가능
ps -o pid,min_flt,maj_flt,rss,comm -p PID
```

관찰	의심
Minor 가 매우 많음	큰 Memory 의 반복 할당 / 해제 → 재사용 , pre-fault
Major 가 계속 발생	Memory 부족 , Swap → free -m, vmstat 1

정리

- Page Fault = 유효하지 않은 PTE 를 만난 Exception. CR2, 같은 명령 재실행
- Handler 는 VMA 를 보고 판단 : 고치거나 , SIGSEGV
- Demand Paging, Zero Page, COW, Stack 확장이 모두 Page Fault
- Minor 는 μs , Major 는 ms + Sleep

다음 : L10. File 을 mmap 해서 배열처럼 읽었다 . read 없이 어떻게 보였는가

L10

mmap 과 Memory Mapped File

mmap 은 File 을 Memory 에 복사하는 함수가 아니다 . VMA 를 만드는 함수다

영상 18 분

▶ 애니메이션 [C2-5 mmap: File ↔ Page Cache ↔ Address Space](#)

문제 상황

```
$ strace ./mmap_cat note.txt
openat(AT_FDCWD, "note.txt", O_RDONLY) = 3
mmap(NULL, 17, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fec1f050000
close(3) = 0
write(1, "hello mmap world\n", 17) = 17
```

read() 가 없다 . 그런데 File 내용이 출력된다

- fd 를 닫은 뒤에도 읽힌다

이미 여러 번 본 mmap

어디에서	누가	종류
execve 가 LOAD Segment 를 올림	Kernel	File, Private
ld.so 가 libc.so.6 을 올림	Loader	File, Private
malloc(4MB), Thread Stack	glibc	Anonymous, Private
shmget / shm_open	Application	Shared

Process 의 주소 공간은 전부 **mmap** 으로 만들어진다

인자 = VMA 의 Field

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

	MAP_PRIVATE	MAP_SHARED
File	쓰면 나만의 복사본 (COW). File 불변	쓰면 File 이 바뀌고 다른 Process 에 보임
Anonymous	0 으로 채워진 나만의 Memory	fork 한 Process 끼리 공유

prot → rwx, flags → p/s, fd·offset → inode·Offset (maps 의 각 열)

첫 접근에서 일어나는 일

p[0] 읽기 → PTE 없음 → Page Fault → VMA: "note.txt 의 Offset 0"
→ Page Cache 에서 Page 를 찾는다 (없으면 Disk 에서)
→ 그 Frame 을 PTE 에 직접 연결 → 명령 재실행

	read()	mmap
Data 경로	Page Cache → 복사 → User Buffer	PTE 가 Page Cache 를 직접 가리킴
비용	System Call + 복사	Page Fault + TLB Miss

mmap: File ↔ Page Cache ↔ Address Space

화면 전환 → [animation.html](#)

읽기 → MAP_SHARED 쓰기 → MAP_PRIVATE 쓰기

▶ 애니메이션 [C2-5 mmap: File ↔ Page Cache ↔ Address Space](#)

MAP_PRIVATE 와 MAP_SHARED 실습

```
$ ./mmap_private
Memory : #ello mmap world
$ cat note.txt
hello mmap world
```

← File 은 그대로

```
$ ./mmap_shared
$ cat note.txt
HELLO MMAP WORLD
```

(write() 호출 없음)
← File 이 바뀜

msync(MS_SYNC): Disk 반영 완료까지 대기 . munmap 과 종료는 반영을 보장하지 않는다

→ Terminal 전환

File 없는 공유 Memory

```
int *shared = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,  
                  MAP_SHARED | MAP_ANONYMOUS, -1, 0);  
fork();
```

private_counter = 0	일반 변수 : COW 로 갈라짐
shared_counter = 100	MAP_SHARED: 같은 Frame

공유는 해 주지만 동기화는 해 주지 않는다 → Chapter 4

주의점 : SIGBUS

1. 두 번째 Page 에 쓰기 성공
2. File 을 4096 Bytes 로 줄였습니다 . Mapping(8192) 은 그대로입니다
Bus error (core dumped)

SIGSEGV	주소가 약속에 없다
SIGBUS	주소는 유효한데 뒤의 실체를 가져올 수 없다

I/O 오류가 return 값이 아니라 **Signal** 로 온다 · 실패 검사는 MAP_FAILED · 크기 0 은 EINVAL

Chapter 2 정리

mmap / brk / execve	→ VMA 를 만든다 (약속)	L06, L10
Memory 접근	→ TLB → Miss 면 Page Table 4 단계	L07, L08
PTE 없음 / 권한 위반	→ Page Fault → VMA 를 보고 판단	L09
Anonymous 첫 접근 / File(Page Cache) / COW / Stack 확장 / SIGSEGV / SIGBUS		
PTE 갱신, 같은 명령 재실행		

Worksheet 20 문제 → Chapter 3: Page Cache 와 Advanced File I/O

다음 시간 · Chapter 3

Page Cache 와 Advanced File I/O

개념 워크시트

[worksheet.html](#)

Lecture 당 10 문항 · 50 문항

애니메이션

[animation.html](#)

5 종

강의 자료

[lecture.html](#)

빈칸 채우기 · 괄호 넣기

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog